

Lab 7 Report: Autonomous Navigation and Waypoint Planning

Course: MCT-454L Mobile Robotics

Student: Anas Gulzar

Date: April 28, 2026

1. Initiating the Gazebo Simulation

To begin, we loaded the TurtleBot3 Burger into the simulated environment by running:

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

This command placed the robot exactly at the center coordinates (0, 0, 0) within the same virtual world we mapped previously during the Lab 5 SLAM exercises.

2. Running Nav2 with the Existing Map

Next, we activated the Nav2 system using the map generated from our previous lab:

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py use_sim_time:=True
map:=$HOME/MR_Lab_MuhammadAnss/Week5/maps/my_map.yaml
```

Executing this fired up essential nodes including the Map Server, the AMCL localization module, and the core planning/control servers. It also launched RViz so we could see the navigation process visually.

3. Calibrating the Starting Position in RViz

Before moving, the robot needed to know its starting location. We used the **2D Pose Estimate** tool in RViz to click on the robot's rough location near the origin. This helped the AMCL system align the live LiDAR scans with the static map walls. We also drove the robot around manually for a short bit so it could perfectly lock in its localization.

Task 1: Basic Point-to-Point Navigation

We tested simple navigation by clicking three different destinations using the **Nav2 Goal** button.

Goal #	Start (x, y)	Destination (x, y)	Orientation (z, w)	Outcome

1	(0.0171, 0.0318)	(1.4513, 0.9929)	(-0.0075, 1.0000)	✓ Success. The robot generated a smooth route and reached the target.
2	(1.4513, 0.9929)	(2.5944, 1.0477)	(-0.7189, 0.6952)	✓ Success. The path planner easily found a way and the controller drove it perfectly.
3	(2.5944, 1.0477)	(2.4980, -0.0532)	(-0.0075, 1.0000)	✓ Success. We saw the local costmap updating rapidly as it squeezed through tight spots.

Note: When we intentionally clicked a goal outside the known map walls, the system correctly rejected it. The global costmap treats unmapped or walled-off areas as restricted zones, making it impossible to plan a path there. We checked the robot's internal position estimate anytime by running `ros2 topic echo /amcl_pose`.

Task 2: Executing a Sequence of Waypoints

For the next phase, we programmed a mission containing 5 specific coordinates. We used the custom waypoints provided for this task:

Waypoint #	X (m)	Y (m)	Z (m)	Yaw (rad)	Notes
1	3.787296	1.233057	0.0	-0.613758	Waypoint 1
2	1.722630	-1.427900	0.0	0.999321	Waypoint 2
3	1.965973	2.495532	0.0	-0.052694	Waypoint 3
4	-0.121979	0.373990	0.0	-0.338357	Waypoint 4
5	2.479727	-0.079857	0.0	-0.612561	Waypoint 5 (Return)

We launched the multi-waypoint script with:

```
ros2 run lab7_nav waypoint_navigator_Task2
```

The robot drove to each of the 5 locations in order. Every time it reached a waypoint, it calculated the next leg of the journey. The Python script behind this uses the `FollowWaypoints` action client to send the entire list of coordinates to the navigation server and monitors the feedback.

Task 3: Passing Waypoints via Command Line

We modified our Python script (`waypoint_navigator_Task3.py`) so we could feed it new coordinates directly from the terminal, avoiding the need to hardcode them. You pass sets of three numbers (X, Y, and Orientation W).

```
ros2 run lab7_nav waypoint_navigator_Task3 -- 0.5 0.0 1.0 1.0 0.5 1.0 0.0 0.0 1.0
```

If you don't type any numbers after the command, it just defaults back to the 5 standard waypoints. The code reads the input arguments and constructs the target list on the fly.

Task 4: Understanding Costmaps

Navigation relies on two main grid maps to avoid crashing:

- **Global Costmap (/global_costmap/costmap)**: This looks at the whole static map we saved. The global path planner uses it to figure out the primary route from start to finish. It doesn't change much unless the main map changes.
- **Local Costmap (/local_costmap/costmap)**: This is a small, moving window centered around the robot. It updates constantly using live sensor data to dodge things that aren't on the static map (like moving objects).

When driving near objects, we noticed the local costmap creates a "bubble" (inflation radius) around the obstacle. This buffer stops the robot from scraping against walls or running into unexpected items.

Task 5: How the Robot Recovers from Trouble

To test the system's resilience, we dropped a virtual box right in front of the robot while it was driving.

1. The LiDAR spotted the box, and the local costmap immediately drew an obstacle zone over it.
2. The robot realized its current path was blocked.
3. It reversed slightly to give itself some breathing room (Backup recovery).
4. It paused to let the costmaps settle (Wait recovery).
5. The global planner quickly drew a brand new route going around the box.
6. The robot followed the new line to the final destination.

These actions are controlled by the `behavior_server`. It can spin the robot to clear bad sensor data, back up, or just wait. It takes commands from the main behavior tree when normal driving fails.

6. What We Saw in RViz

We had several visual layers turned on to monitor the robot:

- **Map (/map)**: The static floor plan.
- **Global/Local Costmaps**: Shown as colored gradients to highlight danger zones.
- **Plans (/plan and /local_plan)**: The long-distance route and the immediate driving arc.
- **AMCL Cloud (/particle_cloud)**: A cluster of arrows showing where the robot thinks it is.
- **Odometry & TF**: Showing movement tracking and frame linkages.

- **LaserScan (/scan):** The raw dots from the LiDAR.

Summary and Conclusions

In this lab, we successfully linked our previous mapping work with active navigation. Here are the main takeaways:

- **Map Utilization:** We proved that the SLAM map from Lab 5 works perfectly as the base for the Nav2 system.
- **Localization is Key:** Without a good starting estimate using AMCL, the robot will get confused and fail to navigate.
- **Smart Planning:** We observed how the global planner handles the big picture while the local controller handles immediate dodging.
- **Mission Automation:** Using the `FollowWaypoints` server allowed us to automate a complex sequence of movements effortlessly.
- **Obstacle Resilience:** Nav2's built-in behavior trees effectively saved the robot from getting permanently stuck when its path was blocked.

Comparing this to Lab 5: SLAM is about exploring and recording an unknown space manually. Navigation is about taking that finished record and letting the machine drive itself intelligently through it.